Second assignment Distributed Computing

Francesco Matano (S5253162), Davide Giusto (S5252480)

December 13, 2024

Contents

1 Introduction **2**
1.1 Delivery 2

2 What questions lead you to this simulator extension? **2**
2.1 Why do you think they are relevant? 2

3 How did you do that? **2**
3.1 Testing 3

4 What is the answer to the original questions you asked? **7**

5 Why is that? What is your interpretation of the results you obtained? **9**

1 Introduction

1.1 Delivery

For the second assignment we must answer to these questions:

- What questions lead you to this simulator extension?
  - Why do you think they are relevant?
- How did you do that?
  - Try to be at the right level of detail: don't include obvious/boring details, and discuss anything that is not obvious.
  - Aim at a document that should be clear to somebody who knows both the original simulator and the programming language.
  - Describe any unexpected, surprising, and interesting things you found.
- What is the answer to the original questions you asked?
  - Why is that? What is your interpretation of the results you obtained?

2 What questions lead you to this simulator extension?

The question we asked ourselves was the following: if we add high-priority peers to the system, which have to store and backup faster than low-priority peers, what will be the impact on the system?

2.1 Why do you think they are relevant?

We think that this question could be relevant because it is not so rare that in real complex system there are nodes that are more important than others and they need their data to be managed more carefully in the event of failures. This can be a useful new function, but we think it is relevant also understand if this can lead to significant worsening in low-priority data losses, which is something that we want to avoid

3 How did you do that?

Note: For all the things that we have done in this assignment we had use only the p2p config file For doing that, we have introduced new elements in the system:

- The percentage on the total peers number of the high-priority nodes (high_priority_per).

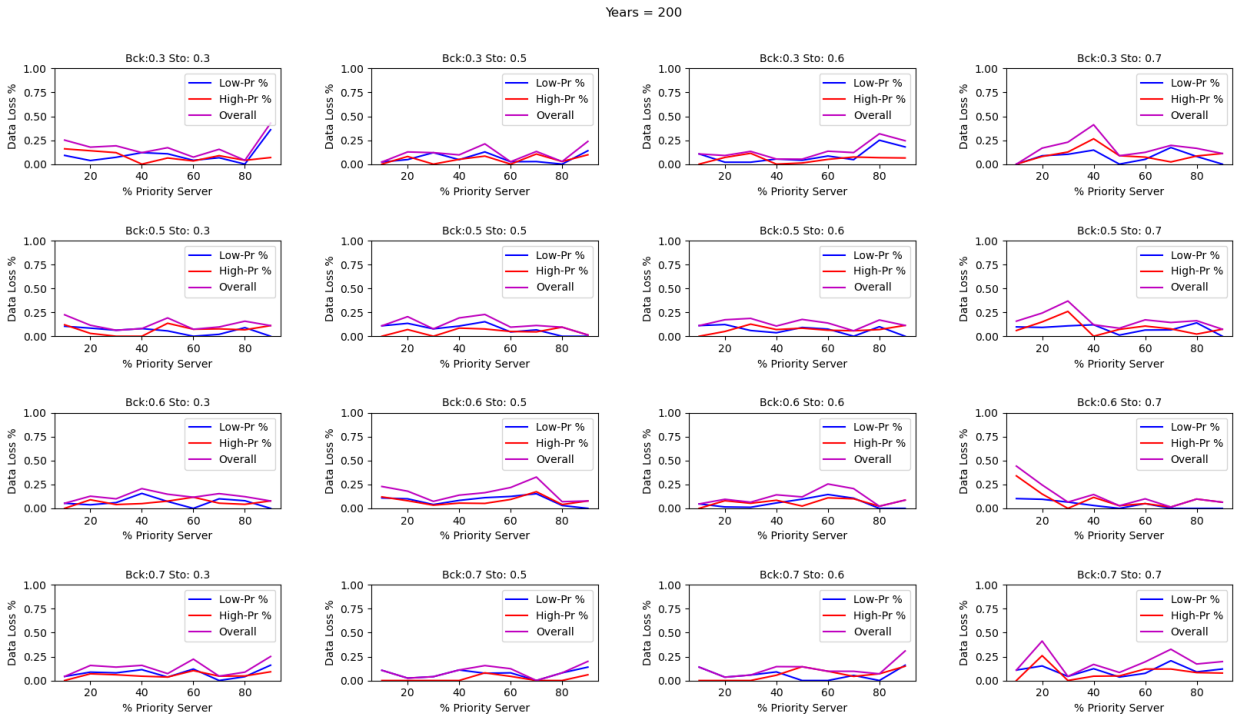
- A counter of the total number of high priority blocks (TOT_BLOCK_PRIORITY).
- A counter of high priority blocks that are already backed up (TOT_BLOCK_BACKUP).
- A counter of the high priority blocks that still we needs to restore (TOT_BLOCK_RESTORE).
- A percentage that is a threshold used to determine if a low-priority peer can restore blocks (PER_RESTORE).
- A percentage that is a threshold used to determine if a low-priority peer can backup blocks (PER_BACKUP).

In order to better explain the last two variables introduced in our extension, we can consider an example: let's suppose that we have PER_BACKUP = 70% and PER_RESTORE = 30%, if a low-priority peer needs to back up one of its block, it cannot go on if the counter of high priority blocks that are already backed up is less than 70% of the total number of high priority blocks. While for the restore process, it cannot go on if the counter of the high priority blocks still to restore is more than 30% of the total number of high priority blocks.

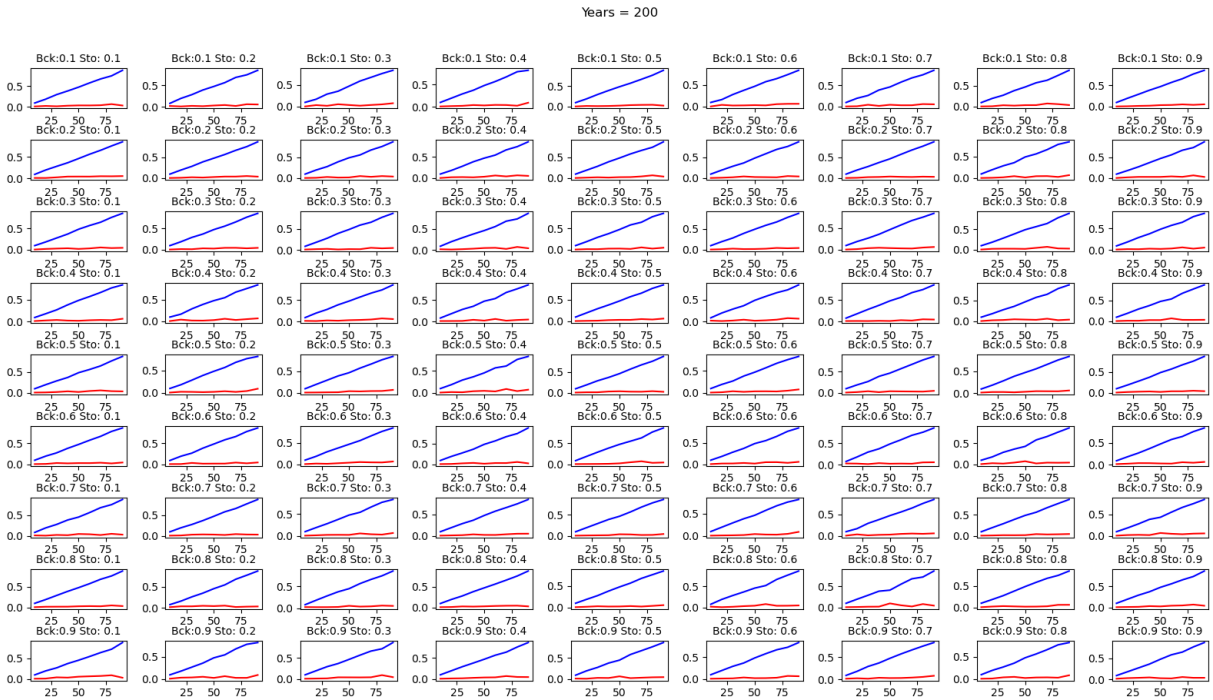
We have also implemented a way to measure data losses in the system for both priority and non-priority peers: whenever a peer has a failure, we count the number of its blocks that are in the network and if they are not enough (equal to k) to recover data, we count the number of the blocks that are lost. We do the same operations also for each nodes whose blocks the failed peer saved. At the end of the execution we will have, for each peers, a counter of the lost blocks and this values will be used to test our new version of the simulator.

3.1 Testing

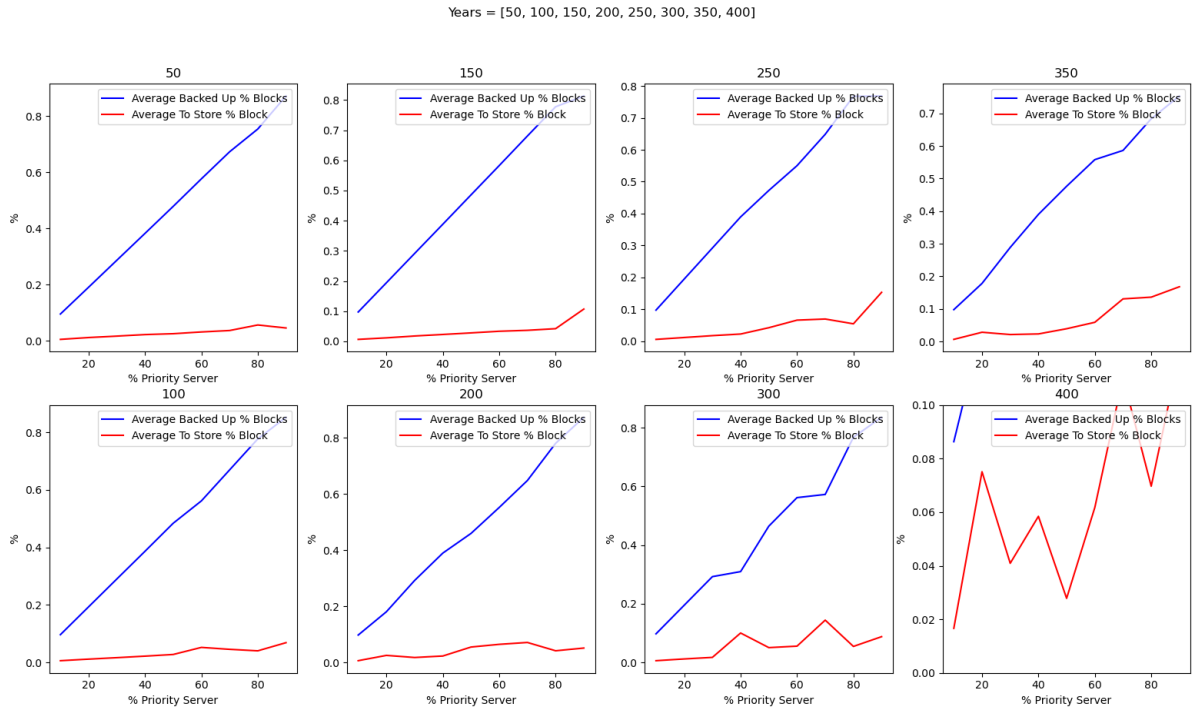
At first, in order to find the best combination of values to assign to PER_RESTORE, PER_BACKUP and high_priority_per, we run multiple times our simulator with all the possible combinations of these values. Surprisingly, we obtained different plots in which almost always the high-priority peers data loss was too high.



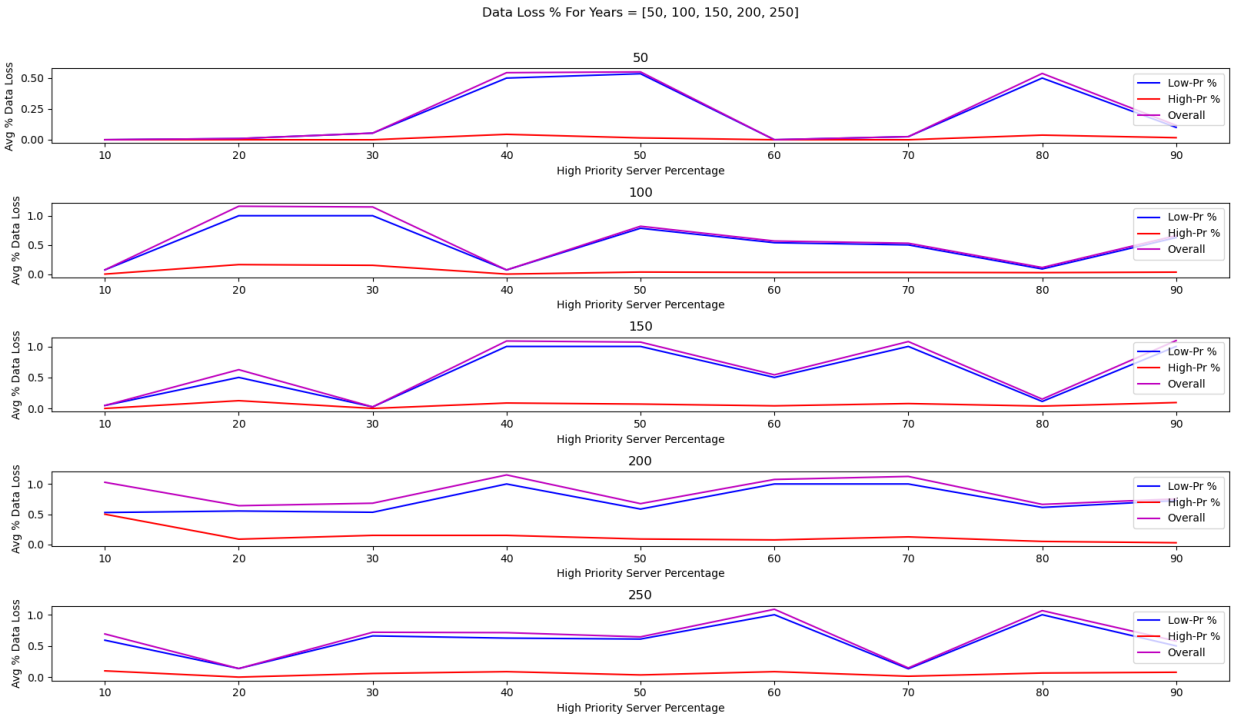
We then thought to check the average percentage of backed up and still to restore blocks of high-priority nodes with all precedent combinations of values and a max-t of 200 years (we also tried with 50 and 100 years), and these are the results that we've obtained:



Observing this plot, we noticed that these average percentages are the same regardless of the combination of values chosen and they can be seen as linear functions. We verified that this is true for every max_t less than 300 years. After this max_t we get a more complex function that is more difficult to resume in a single linear formula with linear computational complexity.

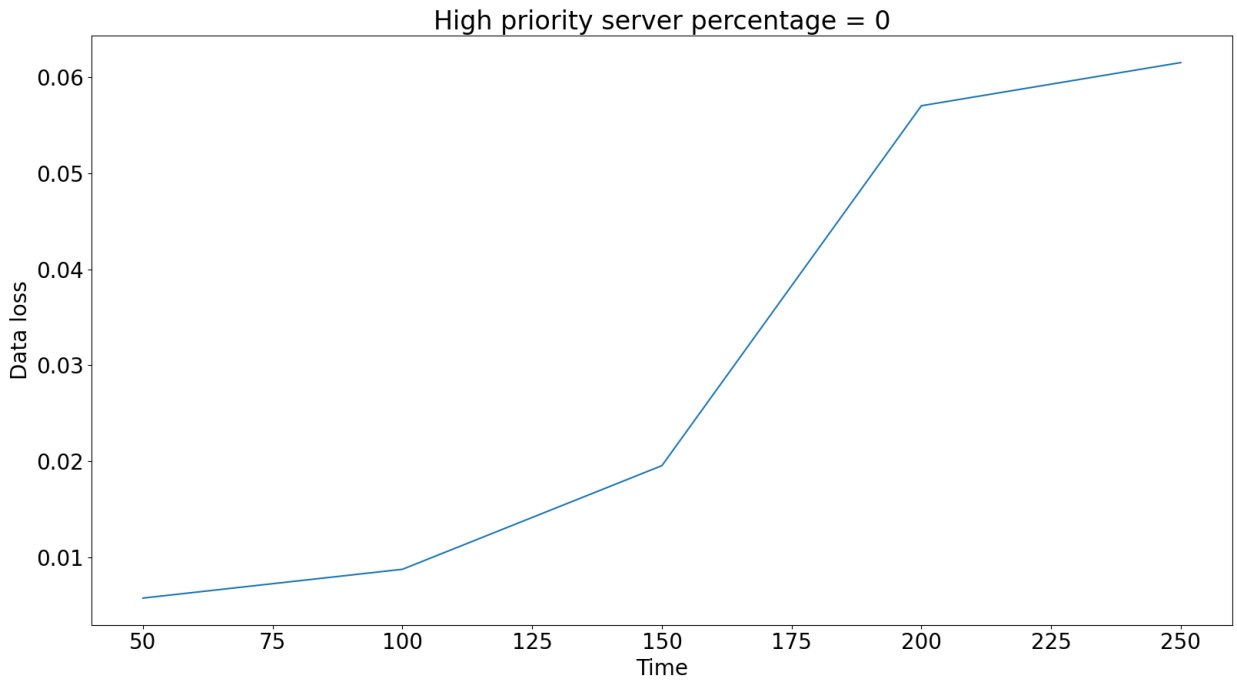


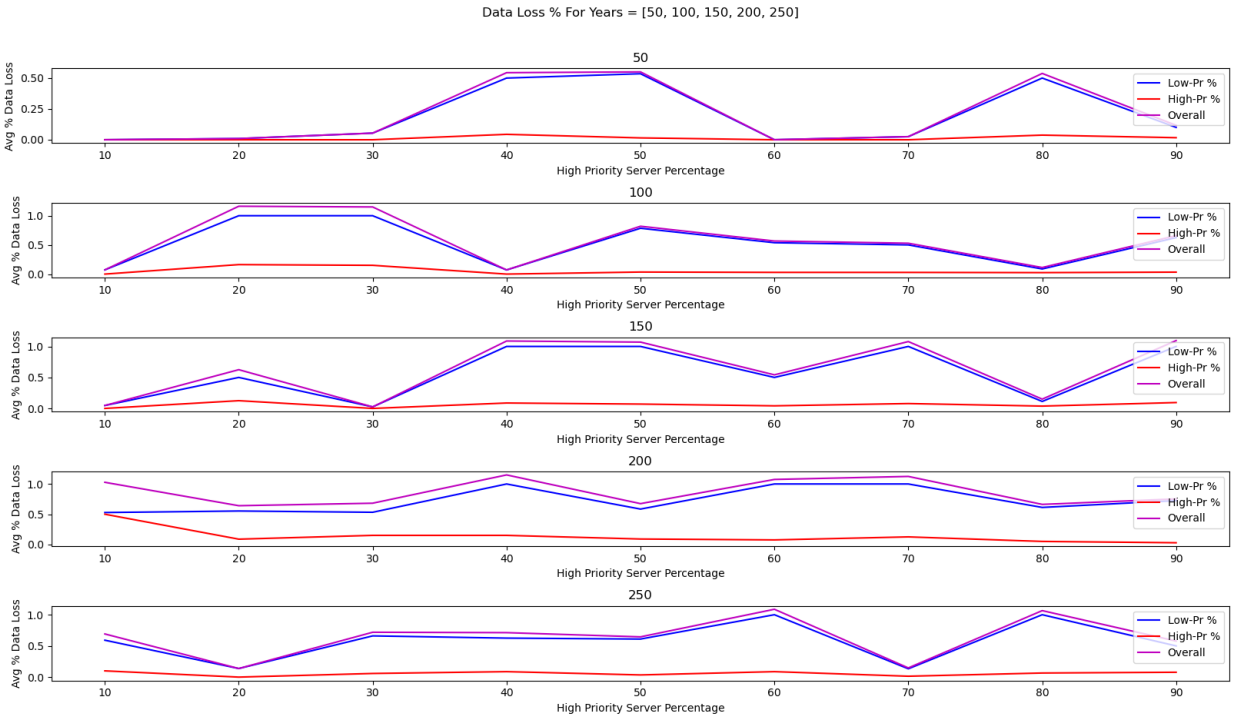
So we then calculated the slopes of this two functions and we use them to respectively calculate prefixed value to assign to PER_BACKUP and PER_RESTORE, and these were the new results that we obtained:



4 What is the answer to the original questions you asked?

What we have discovered is that the new functionality that we introduced in our simulator allow us to better preserve high-priority peer's data but this leads to a worse overall data loss than the system without high-priority peers, as we can see in this two graphics:


High priority server percentage = 0
Data loss
0.06
0.05
0.04
0.03
0.02
0.01
50
75
100
125
150
175
200
225
250
Time



5 Why is that? What is your interpretation of the results you obtained?

This is because Low priority peers allow more of their data to be lost in order to better preserve the data of the high priority peers and this causes an inevitable higher overall loss.
With these values of PER_BACKUP and PER_RESTORE we are privileging too much peers with high priority because we allow high priority peers to backup and restore exactly the number of blocks they need on average.
If instead we want to create a system where there is a greater tradeoff between the privilege of high priority peers and overall data loss of the system, it will be enough to deviate from the average values, as can be observed from the following plots

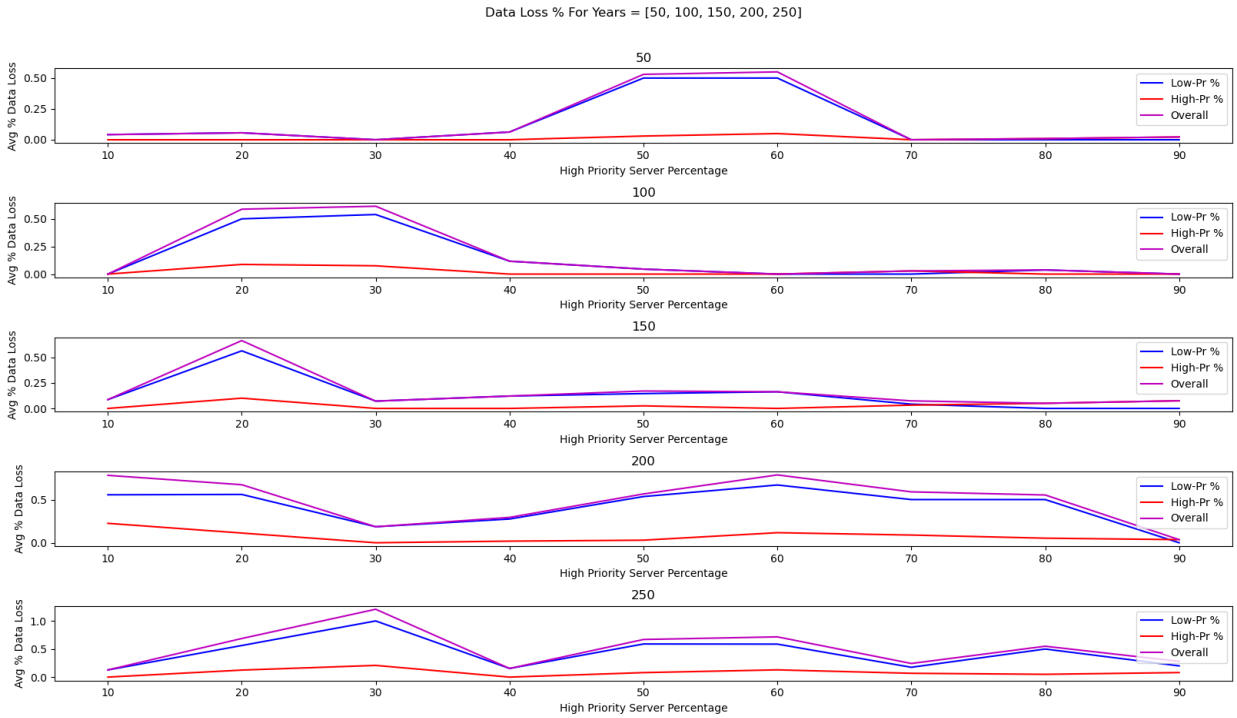


Figure 1: $\frac{1}{2}$ x PER_BACKUP and 2 x PER_RESTORE

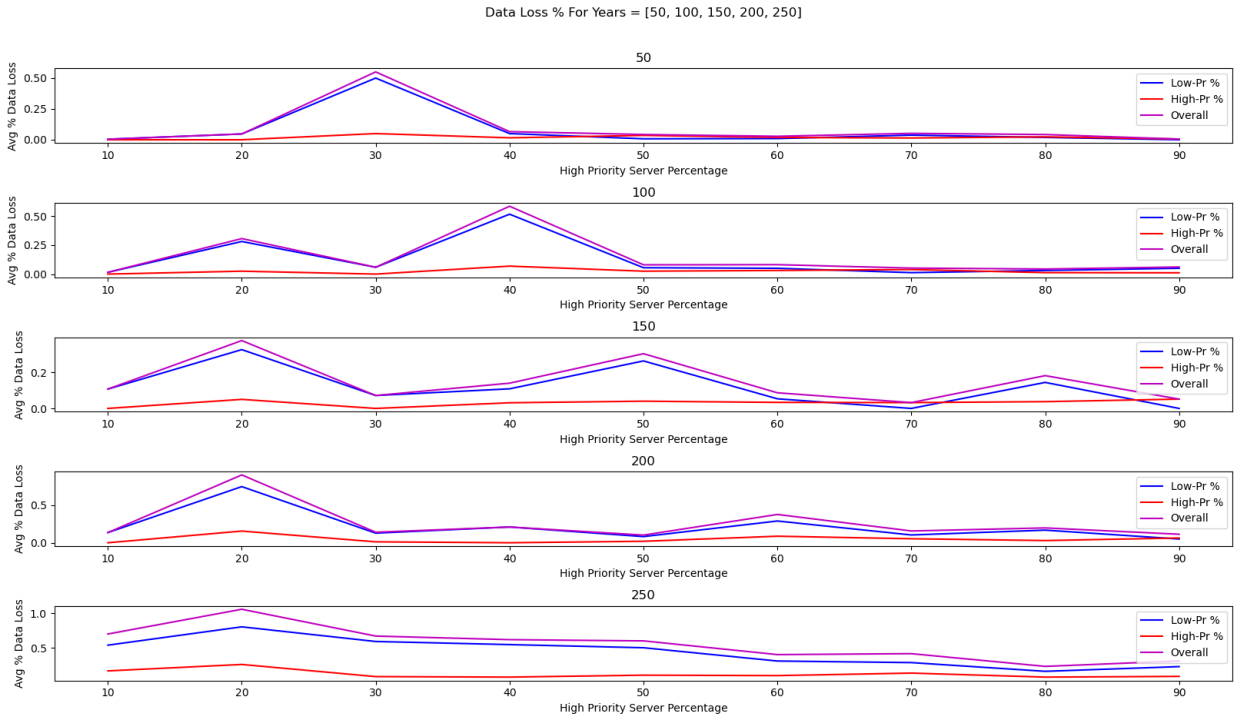


Figure 2: $\frac{1}{4}$ x PER_BACKUP and 4 x PER_RESTORE